

Mois 1 — Fondations Data

Semaine 3 — Analyse et visualisation

Outils : Python · Pandas · Matplotlib · Seaborn · Jupyter

Pourquoi cette étape ?

Tu as maintenant 3 datasets propres et structures.

La semaine 3 marque le passage du nettoyage à l'analyse — de 'quelles données j'ai' à 'que racontent ces données'.

Deux objectifs :

1. Extraire des insights — groupby, aggregations, tris, filtres
2. Communiquer visuellement — bar chart, line chart, histogram, boxplot

Les extractions de dates préparées en semaine 2 (mois, jour_semaine) seront directement exploitées ici.

Commandes de la semaine

Commande	Ce que ca fait	Quand l'utiliser
<code>df.groupby('col')</code>	Regroupe les lignes par valeur d'une colonne	Pour calculer des stats par catégorie
<code>.sum()</code> / <code>.mean()</code> / <code>.count()</code>	Calcule la somme, moyenne, ou nombre	Toujours après un groupby
<code>df.sort_values('col')</code>	Trie le DataFrame par une colonne	Pour classer du plus grand au plus petit
<code>df['col'].value_counts()</code>	Compte les occurrences de chaque valeur	Pour les colonnes textuelles (top produits...)
<code>df[df['col'] > x]</code>	Filtre les lignes selon une condition	Pour isoler un sous-ensemble de données
<code>df.nlargest(n, 'col')</code>	Retourne les n plus grandes valeurs	Pour les top 5, top 10 directement
<code>df.nsmallest(n, 'col')</code>	Retourne les n plus petites valeurs	Pour les valeurs les plus basses
<code>df.reset_index()</code>	Remet l'index a zero apres groupby	Après groupby pour faciliter la visualisation
<code>plt.figure(figsize=(w,h))</code>	Définit la taille du graphique	Avant tout tracé Matplotlib
<code>plt.bar(x, y)</code>	Graphique en barres	Pour comparer des catégories
<code>plt.plot(x, y)</code>	Graphique en ligne	Pour les évolutions dans le temps

<code>plt.hist(col, bins=n)</code>	Histogramme de distribution	Pour voir la distribution d'une variable numérique
<code>sns.barplot(data, x, y)</code>	Barplot avec intervalle de confiance	Version plus lisible que <code>plt.bar</code>
<code>sns.boxplot(data, x, y)</code>	Boîte à moustaches	Pour visualiser la dispersion et les outliers
<code>sns.heatmap(data)</code>	Carte de chaleur	Pour les corrélations entre variables numériques
<code>plt.title() / plt.xlabel()</code>	Titre et labels des axes	Toujours — un graphique sans titre est inutilisable
<code>plt.savefig('nom.png<br ')<="" code=""/></code>	Exporté le graphique en image	Pour sauvegarder dans le dossier visuals/
<code>plt.show()</code>	Affiche le graphique dans Jupyter	En fin de chaque cellule graphique

Les exercices

Exercice 1 — Agrégations avec groupby()

Dataset conseillé : Coffee Sales — colonne coffee_name et money

Pourquoi cet exercice ?

groupby() est la commande la plus utilisée en analyse de données. Elle permet de calculer des statistiques par catégorie — par produit, par mois, par gare. C'est le passage de 'j'ai des données' à 'je comprends mes données'.

Code

```
import pandas as pd

sales = pd.read_csv('coffee_sales_clean.csv')

# Chiffre d'affaires total par produit
ca_par_produit = sales.groupby('coffee_name')['money'].sum()
print(ca_par_produit.sort_values(ascending=False))

# Nombre de ventes par produit
ventes_par_produit = sales.groupby('coffee_name')['money'].count()
print(ventes_par_produit.sort_values(ascending=False))

# Prix moyen par produit
prix_moyen = sales.groupby('coffee_name')['money'].mean()
print(prix_moyen.sort_values(ascending=False).round(2))

# Plusieurs agrégations en une seule fois
resume = sales.groupby('coffee_name')['money'].agg(['sum', 'mean', 'count'])
resume.columns = ['CA total', 'Prix moyen', 'Nb ventes']
print(resume.sort_values('CA total', ascending=False))
```

Les 3 questions

Ce que j'ai fait :

Utiliser groupby() pour calculer des aggregations (somme, moyenne, comptage) par catégorie.

Pourquoi je l'ai fait :

Pour transformer des milliers de lignes individuelles en indicateurs synthétiques par groupe. C'est la base de toute analyse métier.

Ce que ca m'a appris :

groupby() seul ne fait rien — il faut toujours l'enchaîner avec une fonction d'agrégation (.sum(), .mean(), .count()). Le résultat est un nouveau DataFrame qu'on peut trier, filtrer, et visualiser.

Résultat attendu

Un tableau avec les produits et leurs indicateurs. Tu peux voir quel produit génère le plus de chiffre d'affaires, lequel est le plus vendu, et lequel a le prix moyen le plus élevé — ce ne sont pas forcément les mêmes.

Conseil

sort_values(ascending=False) trie du plus grand au plus petit. C'est presque toujours ce qu'on veut pour les analyses métier. round(2) arrondi à 2 décimales pour la lisibilité.

Exercice 2 — Analyse temporelle — ventes par mois

Dataset conseillé : Coffee Sales — colonnes mois et jour_semaine

Pourquoi cet exercice ?

Les colonnes mois et jour_semaine extraites en semaine 2 servent maintenant. L'analyse temporelle est fondamentale en data analyst — comprendre les tendances dans le temps est l'une des premières questions posées par un métier.

Code

```
import pandas as pd

sales = pd.read_csv('coffee_sales_clean.csv')
sales['date'] = pd.to_datetime(sales['date'])
sales['mois'] = sales['date'].dt.month
sales['jour_semaine'] = sales['date'].dt.day_name()

# CA total par mois
ca_par_mois = sales.groupby('mois')['money'].sum().reset_index()
print(ca_par_mois)

# Nombre de ventes par jour de la semaine
ventes_par_jour =
sales.groupby('jour_semaine')['money'].count().reset_index()
print(ventes_par_jour.sort_values('money', ascending=False))

# Quel mois a le CA le plus élevé ?
print('Meilleur mois :', ca_par_mois.loc[ca_par_mois['money'].idxmax()])
```

Les 3 questions

Ce que j'ai fait :

Groupe les ventes par mois et par jour de la semaine pour identifier les tendances temporelles.

Pourquoi je l'ai fait :

Pour répondre à des questions métier concrètes : quel mois est le plus rentable ? Quel jour de la semaine genere le plus de ventes ? Ces insights permettent des décisions opérationnelles.

Ce que ça m'a appris :

Les colonnes temporelles extraites en semaine 2 prennent tout leur sens ici. Sans to_datetime() et dt.month, cette analyse serait impossible. C'est la chaîne complète : nettoyage → extraction → analyse.

Résultat attendu

Un tableau avec le CA par mois et un autre avec les ventes par jour. Tu identifies le mois le plus rentable et le jour le plus actif — ce sont des insights directement exploitables.

Conseil

reset_index() après groupby() transforme le résultat en DataFrame propre avec des colonnes nommées. Sans ça, l'index devient la colonne de groupage et c'est plus difficile à manipuler ensuite.

Exercice 3 — Top 5 et filtrage

Dataset conseillée : Coffee Sales + Transilien

Pourquoi cet exercice ?

En analyse de données, on cherche souvent les extrêmes — les meilleurs produits, les gares les plus fréquentées, les films les mieux notés. `nlargest()` et `value_counts()` sont les outils directs pour ça.

Code

```
import pandas as pd

# --- Coffee Sales : top 5 produits par CA ---
sales = pd.read_csv('coffee_sales_clean.csv')

top5_produits = sales.groupby('coffee_name')['money'].sum().nlargest(5)
print('Top 5 produits par CA :')
print(top5_produits)

# Top 5 produits les plus vendus
top5_ventes = sales['coffee_name'].value_counts().head(5)
print('Top 5 produits les plus vendus :')
print(top5_ventes)

# --- Transilien : top 5 gares par fréquentation ---
voyageurs = pd.read_csv('transilien_clean.csv')

top5_gares = voyageurs.groupby('Nom Gare')['Somme de Montants'].sum().nlargest(5)
print('Top 5 gares par fréquentation :')
print(top5_gares)

# Filtrer uniquement les jours ouvrables
job = voyageurs[voyageurs['Type jour'] == 'JOB']
print('Shape jours ouvrables :', job.shape)
```

Les 3 questions

Ce que j'ai fait :

Identifie les top 5 produits par chiffre d'affaires et les top 5 gares par fréquentation.

Pourquoi je l'ai fait :

Pour répondre à la question métier classique : sur quoi concentrer les efforts ? Les 20% qui font 80% des résultats (loi de Pareto) se trouvent avec ces commandes.

Ce que ça m'a appris :

`nlargest()` est plus direct que `sort_values().head()` — une seule commande fait les deux.
`value_counts()` est la commande la plus rapide pour compter les occurrences dans une colonne textuelle.

Résultat attendu

Deux classements clairs : les 5 produits qui font le plus de chiffre d'affaires sur le e-commerce, et les 5 gares les plus fréquentées sur le Transilien. Avec le filtre JOB tu vois aussi la différence entre jours ouvrables et week-end.

Conseil

Comparez le top 5 par CA et le top 5 par nombre de ventes — ils peuvent être différents. Un produit peu vendu mais cher peut générer plus de CA qu'un produit très vendu mais bon marché. C'est un insight métier important.

Exercice 4 — Premiers graphiques avec Matplotlib

Dataset conseillée : Coffee Sales

Pourquoi cet exercice ?

Matplotlib est la librairie de base de la visualisation Python. Comprendre sa logique (figure, axes, traces, labels) est indispensable avant de passer à Seaborn qui l'utilise en dessous.

Code

```
import pandas as pd
import matplotlib.pyplot as plt

sales = pd.read_csv('coffee_sales_clean.csv')
sales['date'] = pd.to_datetime(sales['date'])
sales['mois'] = sales['date'].dt.month

# --- Bar chart : CA par produit ---
ca_par_produit =
sales.groupby('coffee_name')['money'].sum().sort_values(ascending=False)

plt.figure(figsize=(12, 6))
plt.bar(ca_par_produit.index, ca_par_produit.values, color='steelblue')
plt.title('Chiffre d affaires par produit')
plt.xlabel('Produit')
plt.ylabel('CA total')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('visuals/ca_par_produit.png')
plt.show()

# --- Line chart : CA par mois ---
ca_par_mois = sales.groupby('mois')['money'].sum().reset_index()

plt.figure(figsize=(10, 5))
plt.plot(ca_par_mois['mois'], ca_par_mois['money'], marker='o',
color='steelblue')
plt.title('Evolution du CA par mois')
plt.xlabel('Mois')
plt.ylabel('CA total')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('visuals/ca_par_mois.png')
plt.show()
```

Les 3 questions

Ce que j'ai fait :

Créer un bar chart du CA par produit et un line chart du CA par mois avec Matplotlib.

Pourquoi je l'ai fait :

Pour passer des tableaux de chiffres a une représentation visuelle. Un graphique bien fait communique en un coup d'œil ce qu'un tableau met plusieurs secondes à livrer.

Ce que ca m'a appris :

Matplotlib suit toujours la même logique : 1. définir la taille (figsize), 2. tracer, 3. ajouter titre et labels, 4. afficher ou exporter. Sans titre ni labels, un graphique est inexploitable.

Résultat attendu

Deux graphiques s'affichent dans Jupyter et deux fichiers .png sont exportés dans le dossier visuals/. Le bar chart montre le classement des produits, le line chart montre la tendance mensuelle.

Conseil

`plt.tight_layout()` évite que les labels des axes se chevauchent — utilise-le systématiquement.
`rotation=45` sur les `xticks` est indispensable quand les labels sont longs comme les noms de produits.

Exercice 5 — Visualisations statistiques avec Seaborn

Dataset conseillée : Coffee Sales + Transilien

Pourquoi cet exercice ?

Seaborn produit des graphiques statistiques plus lisibles que Matplotlib avec moins de code. Le boxplot et l'histogramme sont les deux graphiques les plus utiles pour comprendre la distribution d'une variable numérique.

Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

sales = pd.read_csv('coffee_sales_clean.csv')
voyageurs = pd.read_csv('transilien_clean.csv')

# --- Histogramme : distribution des prix ---
plt.figure(figsize=(10, 5))
sns.histplot(sales['money'], bins=20, color='steelblue', kde=True)
plt.title('Distribution des prix des ventes')
plt.xlabel('Prix')
plt.ylabel('Frequence')
plt.tight_layout()
plt.savefig('visuals/distribution_prix.png')
plt.show()

# --- Boxplot : fréquentation par type de jour ---
plt.figure(figsize=(10, 6))
sns.boxplot(data=voyageurs, x='Type jour', y='Somme de Montants')
plt.title('Fréquentation par type de jour (Transilien)')
plt.xlabel('Type de jour')
plt.ylabel('Nombre de voyageurs')
plt.tight_layout()
plt.savefig('visuals/frequentation_type_jour.png')
plt.show()
```

Les 3 questions

Ce que j'ai fait :

Créer un histogramme de la distribution des prix et un boxplot de la fréquentation par type de jour.

Pourquoi je l'ai fait :

Pour visualiser la distribution des données — pas juste la moyenne, mais la forme entière.
L'histogramme montre si les données sont symétriques ou asymétriques. Le boxplot montre les quartiles et les outliers qu'on avait identifiés avec `describe()`.

Ce que ça m'a appris :

Le boxplot visualise exactement ce que `describe()` t'avait donné sous forme de chiffres (25%, 50%, 75%, min, max). C'est la traduction graphique des statistiques descriptives de la semaine 1. La boucle est bouclée.

Résultat attendu

L'histogramme confirme visuellement ce que `describe()` t'avait dit sur les prix — distribution resserrée autour de 31.75. Le boxplot montre clairement la différence de fréquentation entre JOB, SAM et DIM, avec les outliers visibles (heures de pointe).

Conseil

kde=True dans histplot ajoute une courbe de densité par-dessus l'histogramme — elle montre la forme de la distribution plus clairement. Sur des données très asymétriques comme le Transilien, le boxplot sera très étiré vers le haut à cause des heures de pointe.

Exercice 6 — Analyse TMDb — notes et popularité

Dataset conseillée : TMDb Movies

Pourquoi cet exercice ?

Le dataset TMDb permet des analyses plus riches : corrélation entre notes et popularité, distribution des films par année, top films. C'est l'occasion de croiser plusieurs variables et de voir si des tendances se dégagent.

Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

films = pd.read_csv('tmdb_clean.csv')

# Top 10 films les mieux notés
top10 = films.nlargest(10, 'vote_average')[['title', 'vote_average',
'vote_count']]
print('Top 10 films les mieux notés :')
print(top10)

# Distribution des notes
plt.figure(figsize=(10, 5))
sns.histplot(films['vote_average'], bins=20, color='steelblue', kde=True)
plt.title('Distribution des notes (vote_average)')
plt.xlabel('Note moyenne')
plt.ylabel('Nombre de films')
plt.tight_layout()
plt.savefig('visuals/distribution_notes.png')
plt.show()

# Nombre de films par année
films_par_annee = films.groupby('annee')['title'].count().reset_index()
films_par_annee.columns = ['année', 'nb_films']

plt.figure(figsize=(14, 5))
plt.plot(films_par_annee['annee'], films_par_annee['nb_films'],
color='steelblue')
plt.title('Nombre de films par année')
plt.xlabel('Année')
plt.ylabel('Nombre de films')
plt.tight_layout()
plt.savefig('visuals/films_par_annee.png')
plt.show()

# Popularite vs note — scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(films['vote_average'], films['popularity'], alpha=0.3,
color='steelblue')
plt.title('Popularité vs Note moyenne')
plt.xlabel('Note moyenne')
```



```
plt.ylabel('Popularité')
plt.tight_layout()
plt.savefig('visuals/popularite_vs_note.png')
plt.show()
```

Les 3 questions

Ce que j'ai fait :

Analyse la distribution des notes, les films les mieux notés, et l'évolution du nombre de films par année.

Pourquoi je l'ai fait :

Pour pratiquer les agrégations et visualisations sur un dataset différent. TMDB a une structure différente des deux autres — pas de colonne montant mais des variables comme `vote_average` et `popularity` qu'on peut croiser.

Ce que ça m'a appris :

Scatter plot est le graphique pour visualiser la relation entre deux variables numériques. Si les points forment une tendance ascendante, il y a une corrélation positive. Sur TMDB, la relation entre popularité et note n'est pas forcément linéaire — certains films très populaires ne sont pas forcément les mieux notés.

Résultat attendu

Quatre graphiques : distribution des notes (confirme le resserrement vu avec `describe`), top 10 films, évolution du nombre de films par année, et scatter plot popularité vs note. Le scatter plot révèle si les films populaires sont aussi les mieux notés.

Conseil

`alpha=0.3` dans scatter rend les points semi-transparents — indispensable quand il y a beaucoup de points superposés (8820 films). Sans ça, les zones denses sont illisibles.

Exercice de synthèse — Notebook d'analyse complet

Choisis UN dataset et construis un notebook d'analyse de bout en bout qui répond à une question métier précise.

Questions métier suggérées par dataset :

Coffee Sales :

Quels produits et quelles périodes génèrent le plus de chiffre d'affaires ?

Transilien :

Comment la fréquentation varie-t-elle selon le type de jour et la tranche horaire ?

TMDB :

Quels sont les films les mieux notés et y a-t-il une relation entre popularité et note ?

Structure du notebook de synthèse :

1. Chargement du dataset clean
2. Rappel rapide du contexte (cellule Markdown)
3. Au moins 3 aggregations groupby avec interpretations
4. Au moins 3 graphiques (un bar chart, un line chart ou boxplot, un histogramme)
5. Les 3 questions pour chaque commande cle
6. Synthèse finale en Markdown : 3 insights principaux que tu retiens
7. Export de tous les graphiques dans visuals/

Checklist de fin de semaine

Avant de passer a la Semaine 4, tu dois etre capable de :

Utiliser groupby() avec sum(), mean(), count()
Trier et filtrer les résultats d'une aggregation
Identifier les top N valeurs avec nlargest() ou value_counts()
Créer un bar chart et un line chart avec Matplotlib
Créer un histogramme et un boxplot avec Seaborn
Ajouter titre, labels et exporter en .png
Repondre aux 3 questions pour chaque commande
Formuler 3 insights a partir de tes analyses

Semaine 4 — SQL et retransposition

SELECT · WHERE · GROUP BY · ORDER BY · JOIN · PostgreSQL

Retransposer en SQL les analyses faites en Pandas cette semaine